

NiceGUI前端路由与后端路由的实现方法

NiceGUI 的前端路由（基于 `app.router`）和后端路由（基于 `APIRouter`）本质上是对前端页面导航逻辑和后端 HTTP 接口处理逻辑的分层实现，前者依托 NiceGUI 自身的前端渲染引擎 + Starlette 底层封装，后者直接复用 Starlette/FastAPI 的后端路由体系。以下从实现原理、核心 API、进阶用法、底层机制 四个维度，详细阐述二者的实现方法。

一、前端路由（`app.router`）：页面导航的实现

NiceGUI 前端路由的核心目标是 **将 URL 映射到 UI 页面函数**，并实现前端页面的无刷新跳转、参数传递，本质是对浏览器 `history` API + Starlette 路由的封装。

1. 基础实现：路由注册与页面渲染

(1) 核心 API：`@ui.page()` 装饰器（路由注册）

`@ui.page(path)` 是前端路由最核心的注册方式，底层调用 `app.router.add(path, handler)`，将 URL 路径与页面函数绑定。

```
from nicegui import ui, app

# 1. 基础路由注册：URL → 页面函数
@ui.page('/') # 等价于 app.router.add('/', index)
def index():
    ui.label('首页')
    ui.button('跳转到关于页', on_click=lambda: app.router.open('/about'))

# 2. 动态路由：带路径参数的页面（仅支持字符串类型）
@ui.page('/user/{user_id}') # 动态参数 {user_id} 注入页面函数
def user_detail(user_id: str): # 参数名需与路径中的占位符一致
    ui.label(f'用户ID: {user_id}')
    # 解析查询参数（如 /user/123?name=张三）
    name = app.request.query_params.get('name', '未知')
    ui.label(f'用户名: {name}')

# 3. 带参数跳转：传递查询参数/路径参数
@ui.page('/about')
def about():
    ui.label('关于页')
    # 方式1: 跳转到动态路由（路径参数）
    ui.button('查看用户123', on_click=lambda: app.router.open('/user/123'))
    # 方式2: 带查询参数跳转
    ui.button('查看用户456（带姓名）', on_click=lambda: app.router.open('/user/456?name=李四'))

ui.run()
```

(2) 核心机制：页面渲染流程

当用户访问注册的 URL 时，NiceGUI 执行以下步骤：

- 1. Starlette 接收 HTTP 请求，匹配 `app.router` 中注册的路径；
- 2. 调用绑定的页面函数，执行函数内的 UI 组件创建逻辑；
- 3. 将组件渲染为 HTML/CSS/JS（基于 Vue 3 底层），返回给浏览器；
- 4. 浏览器加载页面，实现 UI 渲染。

2. 进阶实现：页面跳转与导航控制

(1) 核心 API： `app.router` 导航方法

方法	作用	示例
<code>app.router.open(path)</code>	跳转到指定 URL（新增历史记录，可回退）	<code>app.router.open('/user/123')</code>
<code>app.router.replace(path)</code>	替换当前历史记录（不可回退）	<code>app.router.replace('/login')</code>
<code>app.router.back()</code>	回退到上一页	<code>app.router.back()</code>
<code>app.router.forward()</code>	前进到下一页	<code>app.router.forward()</code>
<code>app.router.reload()</code>	刷新当前页面	<code>app.router.reload()</code>

(2) 进阶场景：页面间状态传递

前端路由支持两种参数传递方式：

- 方式 1：URL 参数（路径 / 查询参数）：无刷新、可分享（如 `/user/123?name=张三`）；
- 方式 2：全局状态（`app.storage.user`）：传递复杂数据（如对象、列表），不暴露在 URL 中。

```
from nicegui import ui, app

@ui.page('/')
def index():
    # 存储全局状态（用户级，基于 cookie/session）
    app.storage.user['user_info'] = {'id': 123, 'name': '张三', 'age': 20}
    ui.button('跳转到用户详情（全局状态）', on_click=lambda: app.router.open('/user/profile'))

@ui.page('/user/profile')
def user_profile():
    # 获取全局状态
    user_info = app.storage.user.get('user_info', {})
    ui.label(f'用户名: {user_info.get("name")}')
    ui.label(f'年龄: {user_info.get("age")}')

ui.run()
```

3. 底层实现机制

NiceGUI 前端路由的底层依赖两大核心：

1. **Starlette 路由**：处理 HTTP 请求的路径匹配，是后端层面的路由基础；
2. **前端 Vue Router 封装**：NiceGUI 在浏览器端封装了 Vue Router，实现无刷新页面跳转（`app.router.open` 本质是调用 Vue Router 的 `push` 方法）；
3. **状态管理**：`app.storage.user` 基于浏览器 `localStorage/sessionStorage`，实现跨页面状态共享。

二、后端路由（APIRouter）：HTTP 接口的实现

NiceGUI 的 `APIRouter` 是对 Starlette `APIRouter` 的直接封装（兼容 FastAPI 语法），核心目标是**定义 HTTP API 接口**，处理 GET/POST/PUT/DELETE 等请求，返回结构化数据（JSON / 文本 / 文件），无 UI 渲染逻辑。

1. 基础实现：API 路由注册与挂载

(1) 核心 API：APIRouter 实例化 + 装饰器注册

```
from nicegui import APIRouter, ui
from pydantic import BaseModel # 用于请求体校验
from starlette.responses import JSONResponse # 自定义响应

# 1. 实例化 APIRouter（可指定前缀、标签）
api = APIRouter(
    prefix='/api/v1', # 所有接口路径前缀: /api/v1/xxx
    tags=['用户管理'], # 接口文档分组标签
)

# 2. 定义请求体模型（Pydantic 校验）
class UserCreate(BaseModel):
    name: str
    age: int
    email: str | None = None

# 3. 注册 API 路由（支持 GET/POST/PUT/DELETE）
# 方式1: GET 请求（查询参数）
@api.get('/users')
def get_users(page: int = 1, size: int = 10): # 自动解析查询参数
    return {
        'code': 200,
        'data': [{ 'id': 1, 'name': '张三' }, { 'id': 2, 'name': '李四' }],
        'page': page,
        'size': size
    }

# 方式2: GET 请求（路径参数 + 强类型）
@api.get('/users/{user_id}')
def get_user(user_id: int): # 自动转换为 int 类型，非数字会返回 422 错误
    if user_id == 1:
        return { 'id': 1, 'name': '张三', 'age': 20 }
    return JSONResponse(status_code=404, content={ 'code': 404, 'msg': '用户不存在' })
```

```
# 方式3: POST 请求 (请求体 + Pydantic 校验)
@api.post('/users')
def create_user(user: UserCreate): # 自动解析 JSON 请求体, 校验字段
    return {
        'code': 200,
        'msg': '创建成功',
        'data': {'id': 3, 'name': user.name, 'age': user.age}
    }

# 4. 挂载 APIRouter 到 NiceGUI 应用
# 方式1: 通过 ui.run 挂载
ui.run(router=api)

# 方式2: 通过 app.include_router 挂载 (更灵活)
# from nicegui import app
# app.include_router(api)
# ui.run()
```

(2) 核心机制: API 请求处理流程

当客户端调用 API 接口时, NiceGUI 执行以下步骤:

1. Starlette 接收 HTTP 请求, 匹配 `APIRouter` 中注册的路径和方法 (GET/POST 等);
2. 解析请求参数 (路径 / 查询 / 请求体), 自动类型转换 + Pydantic 校验;
3. 调用接口函数, 执行业务逻辑;
4. 将函数返回值转换为 JSON 响应 (默认), 返回给客户端。

2. 进阶实现: 参数校验与自定义响应

(1) 参数校验 (Path/Query/Body)

`APIRouter` 兼容 FastAPI 的参数校验语法, 通过 `Path/Query/Body` 类约束参数规则:

```
from nicegui import APIRouter
from fastapi import Path, Query, Body

api = APIRouter(prefix='/api/v1')

# 路径参数校验: user_id 必须 ≥1, 且描述信息
@api.get('/users/{user_id}')
def get_user(
    user_id: int = Path(..., ge=1, description='用户ID, 必须≥1'),
):
    return {'id': user_id}

# 查询参数校验: name 可选, 长度 ≥2; page 默认为1, ≥1
@api.get('/users')
def get_users(
    name: str | None = Query(None, min_length=2, description='用户名'),
    page: int = Query(1, ge=1, description='页码'),
):
```

```

        return {'name': name, 'page': page}

# 请求体校验: age 必须 ≥0, name 为必填
@api.post('/users')
def create_user(
    name: str = Body(..., description='用户名'),
    age: int = Body(..., ge=0, description='年龄'),
):
    return {'name': name, 'age': age}

```

(2) 自定义响应（状态码 / 响应头 / 文件）

`APIRouter` 支持返回自定义响应（如状态码、响应头、文件下载）：

```

from nicegui import APIRouter
from starlette.responses import JSONResponse, FileResponse, RedirectResponse

api = APIRouter(prefix='/api/v1')

# 自定义状态码 + 响应头
@api.get('/error')
def error_demo():
    return JSONResponse(
        status_code=400,
        content={'code': 400, 'msg': '参数错误'},
        headers={'X-Custom-Header': 'nicegui'}
    )

# 文件下载
@api.get('/download')
def download_file():
    return FileResponse('data.csv', filename='用户数据.csv')

# 重定向
@api.get('/redirect')
def redirect_demo():
    return RedirectResponse('/api/v1/users')

```

3. 底层实现机制

`APIRouter` 的底层完全复用 Starlette/FastAPI 的路由体系：

1. **路由匹配**：基于 Starlette 的 `Router` 类，按路径 + HTTP 方法匹配接口函数；
2. **参数解析**：通过 FastAPI 的 `Depends` 机制，自动解析路径 / 查询 / 请求体参数；
3. **数据校验**：基于 Pydantic 模型，实现请求参数的类型校验和转换；
4. **文档生成**：基于 OpenAPI 规范，自动生成 Swagger 文档（访问 `/docs` 即可查看）。

三、前端路由 + 后端路由：结合使用的最佳实践

实际项目中，前端路由负责页面导航，后端路由负责数据接口，二者结合实现“前端页面 + 后端接口”的完整应用：

```

from nicegui import ui, app, APIRouter
from pydantic import BaseModel

# ----- 后端路由: API 接口 -----
api = APIRouter(prefix='/api')

# 定义请求体模型
class User(BaseModel):
    name: str
    age: int

# 获取用户列表
@api.get('/users')
def get_users():
    return [{'id': 1, 'name': '张三', 'age': 20}, {'id': 2, 'name': '李四', 'age': 25}]

# 创建用户
@api.post('/users')
def create_user(user: User):
    return {'code': 200, 'msg': '创建成功', 'data': {'id': 3, **user.dict()}}

# ----- 前端路由: 页面导航 -----
@ui.page('/')
def index():
    ui.label('用户管理系统').classes('text-2xl font-bold')

# 加载用户列表（调用后端 API）
async def load_users():
    # 前端调用后端 API
    response = await ui.request('/api/users')
    if response.status == 200:
        users = response.json()
        # 渲染用户列表
        for user in users:
            ui.card(
                ui.label(f'ID: {user["id"]}'),
                ui.label(f'姓名: {user["name"]}'),
                ui.label(f'年龄: {user["age"]}'),
            ).classes('mb-2')

# 创建用户（调用后端 API）
async def add_user():
    name = name_input.value
    age = int(age_input.value)
    # 发送 POST 请求
    response = await ui.request(
        '/api/users',
        method='POST',
        json={'name': name, 'age': age}
    )
    if response.status == 200:
        ui.notify('创建成功')
        # 刷新列表

```

```
        await load_users()

# 输入框 + 按钮
name_input = ui.input('用户名')
age_input = ui.input('年龄')
ui.button('创建用户', on_click=add_user)

# 加载用户列表
ui.button('加载用户列表', on_click=load_users)

# ----- 挂载路由并运行 -----
app.include_router(api) # 挂载后端路由
ui.run()
```

四、核心差异与底层总结

维度	前端路由 (app.router)	后端路由 (APIRouter)
核心依赖	Vue 3 前端路由 + Starlette 基础路由	Starlette APIRouter + FastAPI 校验 / 文档
数据返回	渲染 UI 组件 (HTML/CSS/JS)	返回结构化数据 (JSON / 文本 / 文件)
参数处理	仅支持字符串路径参数，需手动解析查询参数	强类型参数解析，自动校验 (Path/Query/Body)
导航能力	支持页面跳转 (open/back/replace)	无导航能力，仅响应 HTTP 请求
文档支持	无	自动生成 Swagger/OpenAPI 文档 (/docs)
适用场景	多页面 UI 应用	后端 API 接口开发

关键结论

- 前端路由是**面向用户的页面导航层**，核心是“URL → UI 页面”的映射，依赖前端渲染引擎；
- 后端路由是**面向数据的接口处理层**，核心是“HTTP 请求 → 数据响应”的映射，依赖 Starlette/FastAPI 底层；
- 二者结合是 NiceGUI 开发完整 Web 应用的标准模式：前端路由管理页面，后端路由提供数据接口，前端通过 `ui.request` 调用后端接口。

通过以上实现方法，你可以基于 NiceGUI 快速构建“前端页面 + 后端接口”的全栈 Web 应用，无需额外搭建 FastAPI/Starlette 服务，实现一站式开发。

ui.page 与 APIRouter 的关系深度解析

在 NiceGUI 中，`ui.page` 和 `APIRouter` 是处理路由的两大核心工具，但定位、用途和底层实现差异显著。本文将从核心定位、底层关联、使用场景、交互方式等维度，全面解析二者的关系，帮助你精准区分和协同使用。

一、核心定位与本质差异

维度	ui.page	APIRouter
核心用途	定义「前端页面路由」，返回可视化 UI 页面 (HTML 渲染)	定义「后端 API 路由」，返回结构化数据 (JSON/XML 等)
目标受众	终端用户 (浏览器渲染页面)	客户端程序 (前端 JS / 其他服务调用)
底层依赖	基于 NiceGUI 自身的页面注册机制，底层封装 Starlette 路由	直接基于 Starlette 的 APIRouter，与 FastAPI 路由逻辑一致
返回值类型	无显式返回值 (通过 ui.xxx 构建 DOM)	显式返回 dict/Response (序列化为 JSON)
路径匹配优先级	低于 APIRouter (同路径时 APIRouter 优先)	高于 ui.page (同路径时优先匹配)

1. ui.page：前端页面的路由入口

ui.page 是 NiceGUI 专为构建交互式前端页面设计的装饰器，核心作用是将一个函数注册为「页面处理器」：

- 函数内通过 ui.label、ui.button、ui.card 等组件构建页面 DOM；
- 访问对应路径时，NiceGUI 会渲染该函数定义的 UI 并返回 HTML 响应；
- 支持路径参数 (如 @ui.page("/user/{user_id}"))，但主要用于页面渲染，而非数据交互。

示例：

```
from nicegui import ui

# 前端页面路由：访问 /home 时渲染页面
@ui.page("/home")
def home_page():
    ui.label("欢迎来到首页").classes("text-3xl")
    ui.button("点击测试", on_click=lambda: ui.notify("点击成功"))

ui.run()
```

2. APIRouter：后端 API 的路由管理器

APIRouter 是 NiceGUI 对 Starlette APIRouter 的直接封装，核心作用是模块化管理后端 API 接口：

- 专注于「数据交互」，返回 JSON 等结构化数据，不渲染 UI；
- 支持 GET/POST/PUT/DELETE 等标准 HTTP 方法；
- 可通过中间件实现权限校验、日志、跨域等后端逻辑；
- 本质是「纯接口路由」，与前端页面解耦。

示例：


```
from nicegui import APIRouter, app

api = APIRouter(prefix="/api")

# 后端 API 路由: 访问 /api/hello 时返回 JSON
@api.get("/hello")
def hello_api(name: str = "Guest"):
    return {"message": f"Hello {name}!"}

app.include_router(api)
```

二、底层关联：共享 Starlette 路由系统

NiceGUI 基于 Starlette (ASGI 框架) 构建, `ui.page` 和 `APIRouter` 最终都会注册到 Starlette 的「路由注册表」中, 这是二者的核心关联点:

1. 路由注册流程:

- `ui.page`: 装饰器会将页面函数封装为 Starlette 的 `Route` 对象, 注册到 `app` 的路由列表;
- `APIRouter`: 通过 `app.include_router()` 将子路由批量注册到 `app` 的路由列表。

2. 请求处理流程:

当浏览器 / 客户端发起请求时, Starlette 会按「路由注册顺序 + 路径匹配规则」找到对应的处理器:

- 匹配到 `APIRouter` 定义的 API 路由 → 执行 API 函数, 返回 JSON 响应;
- 匹配到 `ui.page` 定义的页面路由 → 执行页面函数, 渲染 UI 并返回 HTML 响应;
- 无匹配 → 返回 404 错误。

关键细节：路由优先级

若 `ui.page` 和 `APIRouter` 定义了**完全相同的路径** (如 `/test`) , `APIRouter` 会优先匹配, 因为:

- `APIRouter` 的路由通过 `app.include_router()` 注册, 通常早于 `ui.page` (若 `ui.page` 定义在其后, 顺序会反转) ;
- Starlette 路由匹配遵循「先注册先匹配」原则, 因此需**严格避免路径冲突**。

示例 (冲突场景) :

```
from nicegui import ui, APIRouter, app

# 1. 注册 API 路由 (路径 /test)
api = APIRouter()
@api.get("/test")
def api_test():
    return {"type": "api"}
app.include_router(api)

# 2. 注册页面路由 (路径 /test)
@ui.page("/test")
def page_test():
```

```
ui.label("page")
```

```
# 访问 /test → 响应 {"type": "api"} (API 优先)
ui.run()
```

三、协同使用场景（核心价值）

`ui.page` 和 `APIRouter` 并非互斥，而是**互补**的，典型协同场景如下：

场景 1：前后端分离（同一项目内）

前端页面（`ui.page`）通过 AJAX/websocket 调用后端 API（`APIRouter`）获取数据，实现交互。

示例：

```
from nicegui import ui, APIRouter, app

# 1. 后端 API (APIRouter)
api = APIRouter(prefix="/api")
@api.get("/user/{user_id}")
def get_user(user_id: int):
    # 模拟从数据库获取用户数据
    return {"id": user_id, "name": f"用户{user_id}", "age": 20}
app.include_router(api)

# 2. 前端页面 (ui.page)
@ui.page("/user/{user_id}")
def user_page(user_id: str):
    ui.label("用户详情").classes("text-2xl")
    # 调用后端 API 获取数据
    user_data = ui.run_javascript(f"fetch('/api/user/{user_id}').then(r => r.json())")
    # 渲染数据
    ui.json(user_data).classes("mt-4")

ui.run()
```

场景 2：多模块拆分（大型项目）

将「页面路由」和「API 路由」按业务模块拆分，各自独立管理：

```
project/
├─ main.py           # 入口：挂载路由 + 启动应用
├─ pages/            # 页面模块 (ui.page)
│   ├─ home.py
│   └─ user.py
└─ api/              # API 模块 (APIRouter)
    ├─ user_api.py
    └─ order_api.py
```

示例（模块化协同）：

```

# main.py
from nicegui import app, ui
from pages.home import home_page
from pages.user import user_page
from api.user_api import user_api
from api.order_api import order_api

# 挂载 API 路由
app.include_router(user_api)
app.include_router(order_api)

# 注册页面路由（ui.page 无需显式挂载，装饰器自动注册）
ui.run()

# pages/user.py
from nicegui import ui
@ui.page("/user/{user_id}")
def user_page(user_id: str):
    # 调用 /api/user/{user_id} 获取数据并渲染
    ...

# api/user_api.py
from nicegui import APIRouter
user_api = APIRouter(prefix="/api/user")
@user_api.get("/{user_id}")
def get_user(user_id: int):
    ...

```

场景 3：API 供外部调用 + 内置调试页面

通过 `APIRouter` 提供对外的标准化 API，同时通过 `ui.page` 构建内置调试页面，方便测试 API：

```

from nicegui import ui, APIRouter, app

api = APIRouter(prefix="/api/order")
@api.post("/create")
def create_order(data: dict):
    return {"order_id": 123, "status": "created"}
app.include_router(api)

# 内置调试页面：模拟调用创建订单 API
@ui.page("/debug/order")
def debug_order():
    with ui.card():
        ui.label("创建订单调试").classes("text-xl")
        name_input = ui.input("商品名称")
        btn = ui.button("创建订单", on_click=lambda: create_order_click(name_input.value))
        result = ui.json({})

    async def create_order_click(name: str):
        # 调用内部 API
        response = await app.request_client.post(

```

```
        "/api/order/create",
        json={"goods_name": name}
    )
    result.set_content(await response.json())

ui.run()
```

四、关键区别总结表

特性	ui.page	APIRouter
核心目标	渲染交互式前端页面	提供后端数据接口
返回内容	HTML 页面（含 NiceGUI 组件）	JSON/Response（结构化数据）
HTTP 方法支持	仅 GET（页面访问默认 GET）	全量支持（GET/POST/PUT/DELETE 等）
参数解析	仅路径参数（/{param}）	路径 / 查询 / 请求体参数全支持
中间件支持	仅全局中间件（app.add_middleware）	全局 / 路由级中间件（add_middleware）
跨域处理	无需（同域访问）	需配置 CORS 中间件（跨域调用时）
复用性	仅在 NiceGUI 项目内有效	可被任意客户端调用（通用 API）
调试方式	浏览器直接访问页面	工具（Postman/curl）调用接口

五、最佳实践

1. 路径规划：避免冲突

为 API 和页面路由设置**差异化前缀**，例如：

- API 路由：统一前缀 `/api/v1`（如 `/api/v1/user`）；
- 页面路由：无特殊前缀（如 `/user`）或前缀 `/page`（如 `/page/user`）。

2. 职责分离

- `ui.page`：只负责「页面渲染 + 前端交互逻辑」（如按钮点击、表单提交）；
- `APIRouter`：只负责「后端业务逻辑」（如数据校验、数据库操作、权限校验）。

3. 数据交互规范

- 前端页面调用 API 时，使用 `ui.run_javascript()`（客户端请求）或 `app.request_client`（服务端内部请求）；
- API 返回统一格式（如 `{"code": 200, "data": {...}, "msg": "success"}`），便于前端统一处理。

4. 中间件使用

- 全局中间件（如 CORS、日志）：通过 `app.add_middleware()` 添加；
- API 专属中间件（如权限校验）：通过 `APIRouter.add_middleware()` 添加；
- 页面路由无需专属中间件（前端权限通过 API 校验实现）。

六、常见误区

误区 1：用 `ui.page` 实现 API

部分开发者会在 `ui.page` 函数中返回 JSON，这是错误的：

```
# 错误示例
@ui.page("/api/error")
def wrong_api():
    return {"message": "error"} # 返回值会被忽略，页面仍渲染空 HTML
```

原因：`ui.page` 的返回值无意义，NiceGUI 会强制渲染 HTML 响应，而非 JSON。

误区 2：用 `APIRouter` 渲染页面

`APIRouter` 函数返回 HTML 字符串无法替代 `ui.page`，因为缺少 NiceGUI 的组件生命周期和交互能力：

```
# 不推荐
@api.get("/wrong-page")
def wrong_page():
    return "<h1>Hello</h1>" # 无 NiceGUI 组件交互能力
```

误区 3：忽略异步特性

- `ui.page` 函数支持 `async def`，建议与 `APIRouter` 异步函数配合，提升并发性能；
- API 调用耗时操作（如数据库查询）时，必须用异步（`async/await`），避免阻塞事件循环。

七、总结

`ui.page` 和 `APIRouter` 是 NiceGUI 路由体系的「一体两面」：

- **关联**：共享 Starlette 底层路由系统，最终都注册到 app 实例；
- **差异**：前者聚焦前端页面渲染，后者聚焦后端 API 提供；
- **协同**：通过「页面调用 API」实现前后端一体化开发，是 NiceGUI 构建复杂应用的核心模式。

掌握二者的分工与协同，能让你在 NiceGUI 项目中既快速构建交互式页面，又能提供标准化、可复用的后端 API，兼顾开发效率和系统扩展性。